

Prototype write up

Template

This project is based on Template 4.1: Orchestrating AI models to achieve a goal.

Overview of the project

This project is a privacy-focused tool that helps individuals develop self-awareness of their emotions. A user watches a short emotional stimulus while their facial expressions are analysed in real time; their spoken responses are then transcribed and analysed for tone; a local language model synthesises this into a personalised psychological profile with wellbeing recommendations. All processing runs locally with zero data transmission, targeting people who cannot afford therapy or want an impersonal first step. It is not a replacement for professional care. The project is based on Template 4.1: Orchestrating AI models to achieve a goal, combining four pre-trained models across four domains: vision (DeepFace), audio (Whisper), text (sentiment analysis), and language generation (Ollama).

The previous prototype implemented the vision component (real-time facial emotion detection, ~127 FPS, ~73ms latency). This prototype implements the audio (speech-to-text) component and the project's testing framework.

Features being implemented in the prototype.

Refactoring the vision prototype into a pipeline module

The facial-emotion prototype from the previous iteration was originally a single flat script (`emotion_test.py`) in which every statement, webcam capture, the analysis loop, CSV export, and a call to the Ollama language model, executed at the top level of the file. The central goal of Template 4.1 is to *orchestrate several independent models* so this structure, although sufficient for a feature prototype, is not appropriate for the full pipeline.

To address this, the file was copied into the main project and renamed `emotion_detector.py`, and refactored in three ways. First, the runnable logic was wrapped in a `run_session()` function that returns the collected emotion timeline instead of acting on it directly, so that importing the module only *defines* the capability instead of *executing* it. Second, an `if __name__ == "__main__":` guard was added, allowing the file to be run directly for isolated testing while remaining safely importable by other modules. Third, the Ollama call was removed entirely, leaving `emotion_detector.py` responsible for vision alone; language generation now belongs to the orchestration layer.

This refactoring also made the module testable. A pytest suite (`test_emotion_detector.py`) unit-tests the two functions that don't depend on hardware: `get_current_scene()` and `summarise_timeline()`. Both are deterministic, so they can be checked with fixed inputs and expected outputs, including edge cases. The webcam capture in `run_session()` is not unit-tested, since its live input can't be reproduced deterministically; it is verified through feature testing instead.

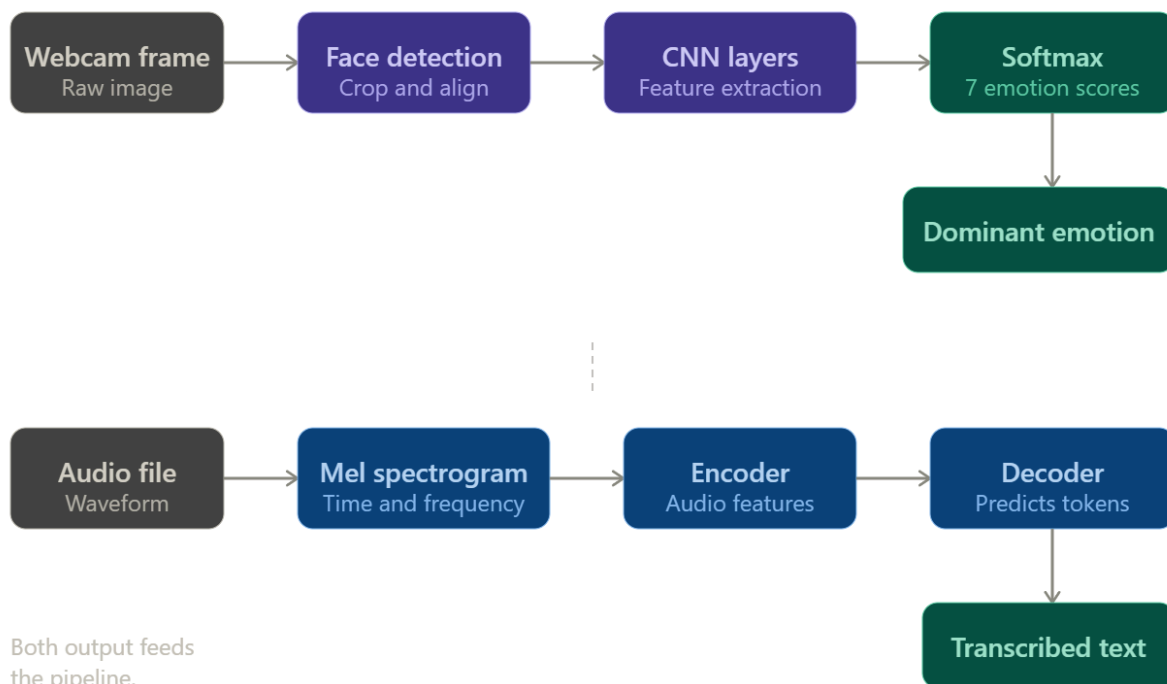
Speech-to-text component

The audio stage of the pipeline was implemented in a new module, `transcriber.py`, which converts a recorded audio file into text. It is built on `faster-whisper`, a reimplementation of OpenAI's Whisper that runs roughly four times faster on CPU at the same accuracy and supports int8 quantisation to reduce memory use, which matters because the project runs locally on consumer hardware with no GPU. The component is structured as a `Transcriber` class: the model is loaded once when the object is created and reused for every subsequent transcription, avoiding the cost of reloading it on each call. A `transcribe()` method takes an audio file, runs it through the model, and returns the recognised text along with the detected language and audio duration. For this iteration the input is a pre-recorded file; live microphone capture is planned for a later stage. As with the vision module, the heavy dependency is imported only when the model is actually instantiated, so the module can be imported without `faster-whisper` present, keeping it consistent with the project's modular structure.

Creating the pipeline

A new `pipeline.py` module was created to act as the orchestrator. It imports `emotion_detector`, calls its `run_session()` function to capture a session and receive the emotion timeline, and then passes that timeline to `summarise_timeline()` to obtain the dominant emotion per scene. The pipeline is structured so that the remaining models, speech-to-text, sentiment analysis, and the Ollama language model, can be added as further stages without changing the individual model modules. This keeps each model self-contained while `pipeline.py` remains solely responsible for sequencing them and passing data between stages.

Explanation of the algorithms, techniques and methods



Both pre-trained models used in this iteration are deep neural networks, but they operate on different data types and solve different problems. The diagram shows how each transforms its input into a usable output.

Facial emotion recognition (DeepFace)

DeepFace classifies emotion from an image using a convolutional neural network (CNN). A webcam frame is first passed to a face detector that crops the face. The cropped face goes through convolutional layers, which slide filters over the image to pick out patterns (edges, then shapes, then features like a smile or a raised brow), and pooling layers, which shrink the data to keep only the strongest signals. A final softmax step turns the network's scores into probabilities across seven emotions and returns the highest one. Each frame is analysed on its own with no memory of previous frames, which is why neutral or ambiguous faces are often misread.

Speech-to-text (Whisper)

Whisper is a transformer model that converts audio to text. The audio is first turned into a mel spectrogram, a picture of how its frequencies change over time. An encoder compresses this into internal features, and a decoder then produces the transcript one token at a time, predicting each next word from the audio and the words so far. Because it models language and not just sounds, it handles continuous speech and punctuation well. The faster-whisper version used here runs the same model more efficiently, giving the same accuracy at lower latency and memory.

Code explanation

The transcriber is built as a class so the Whisper model is loaded only once and then reused:

```
class Transcriber:
    def __init__(self, model_size="base"):
        from faster_whisper import WhisperModel
        self.model = WhisperModel(model_size, device="cpu", compute_type="int8")
```

Loading a Whisper model is slow, so doing it in `__init__` means the cost is paid a single time when the object is created, not on every transcription. The settings `device="cpu"` and `compute_type="int8"` are chosen for the target hardware: the machine has no GPU, and int8 quantisation lowers memory use and speeds up inference on CPU.

The import of `faster_whisper` sits *inside* `__init__` rather than at the top of the file. This is deliberate: it means the module can be imported by the pipeline without the heavy library being installed, and the dependency is only required when a `Transcriber` is actually built. The vision module uses the same pattern for its OpenCV and DeepFace imports.

The transcription itself is one short method:

```
def transcribe(self, audio_path, language="en"):
    segments, info = self.model.transcribe(audio_path, language=language, beam_size=5)
    text = " ".join(seg.text.strip() for seg in segments).strip()
    return Transcription(text=text, language=info.language, duration=info.duration)
```

Note that faster-whisper returns segments as a generator, so no transcription actually happens until the segments are iterated. The `" ".join(...)` line is what triggers the work, stitching the per-segment pieces into a single clean string. That string is returned inside a small `Transcription` object alongside the detected language and audio duration, so the rest of the pipeline gets the text in a predictable form.

Visual representation of the product or results

Placeholder, nothing captured yet. To add: (1) screenshot of pytest passing, (2) screenshot of Whisper transcribing sample.wav, (3) emotion timeline graph from the preliminary report.

Evaluation and future improvements

This iteration reaches its main goal: turning a single proof-of-concept script into a modular system. The vision prototype became an importable component, a pipeline module was added to coordinate the models, and the speech-to-text model was added in the same style. A pytest framework was also set up, with passing unit tests on the vision logic.

The main strength is separation of concerns: each model sits in its own module with a clean interface, and isolating the pure logic from the model and hardware code is what made unit testing possible.

A weakness is that the speech-to-text component has not yet been run on the target hardware, so its real latency, memory, and accuracy are unmeasured. It is also not yet wired into the pipeline.

Next steps: benchmark the transcriber on a sample file (with a lighter model like Vosk as fallback if needed), integrate it into the pipeline, then add the sentiment and Ollama stages the same way, each tested where its data passes to the next stage.